



PECE Programa de
Educação Continuada

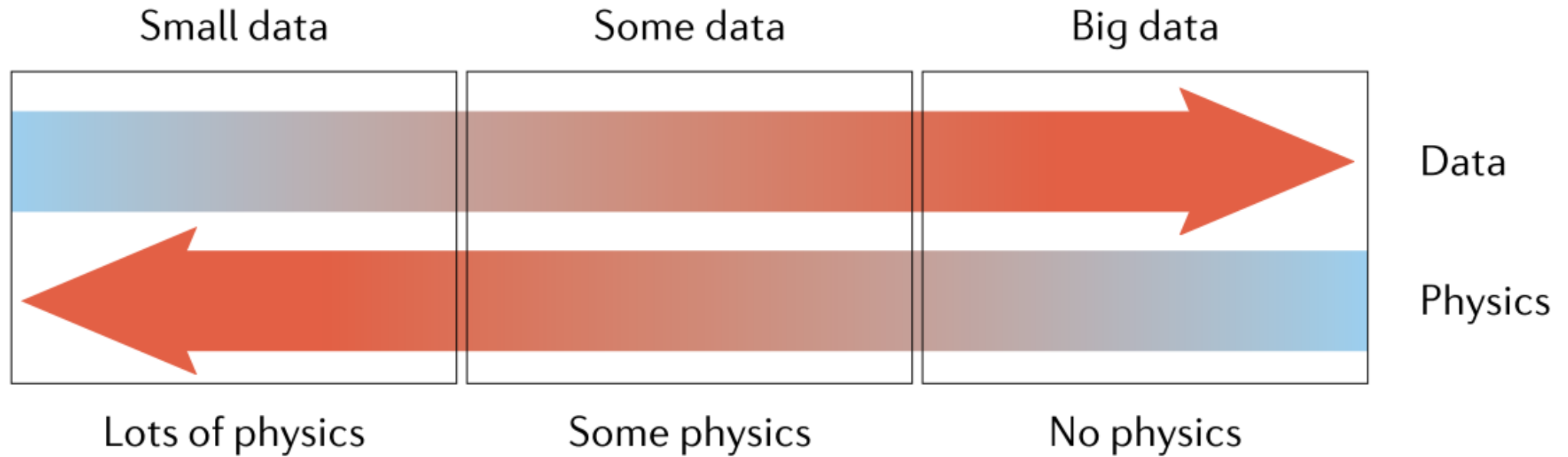
Escola Politécnica da USP

Introdução a Redes Neurais

Marlon Sproesser Mathias

Aula 6

Physics-Informed Neural Networks



KARNIADAKIS, G. E. et al. Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021.

Physics-Informed Machine Learning

O que é?

PIML

Formas de se fazer

Física $\leftarrow$ Dados

- Usar séries de dados para corrigir soluções físicas
- Melhorar as aproximações usadas na física com séries de dados

Dados $\leftarrow$ Física

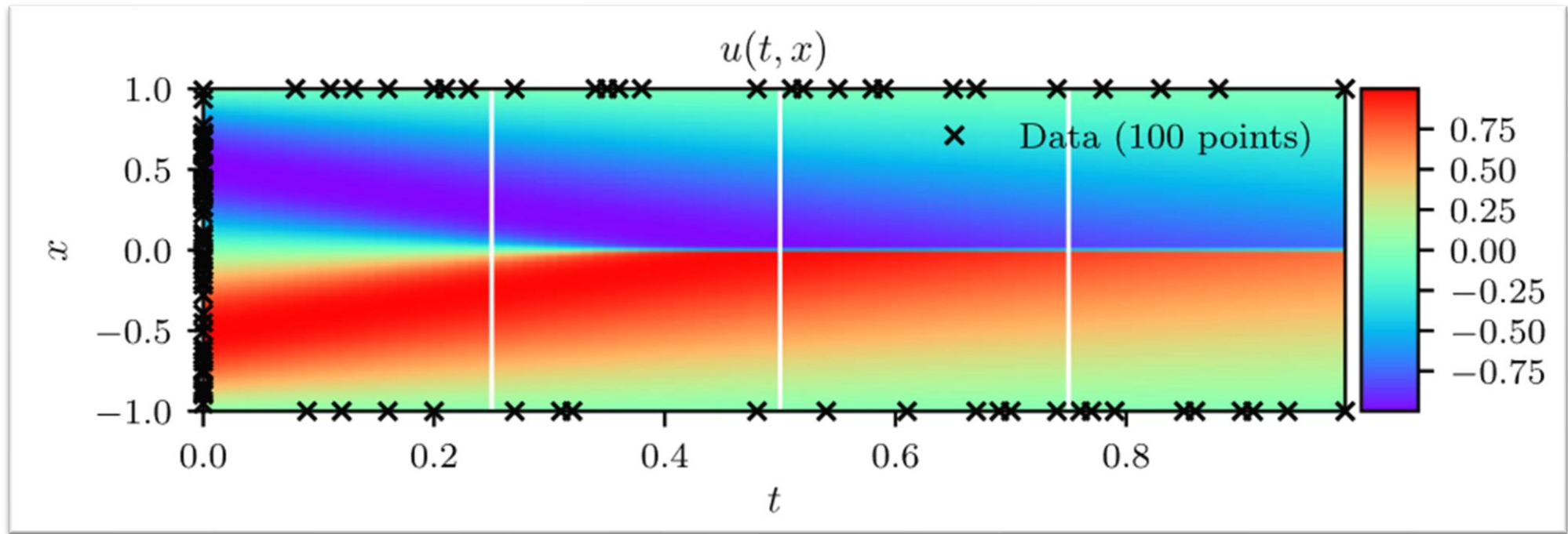
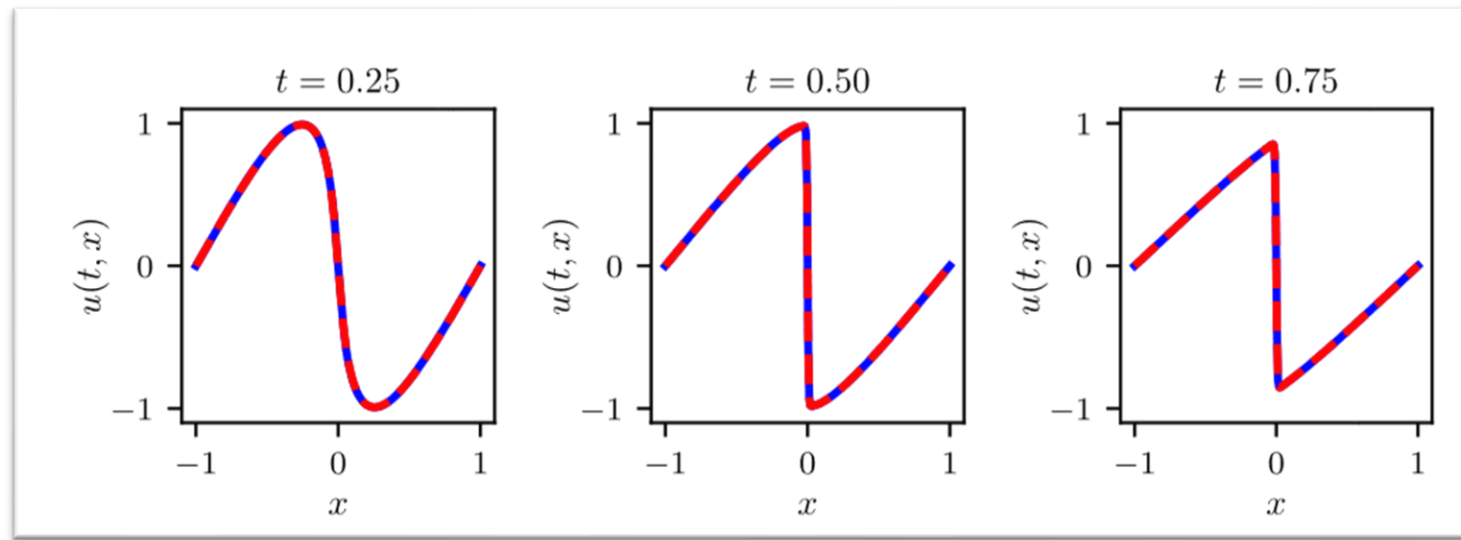
- Adicionar restrições físicas à modelagem por dados
- Usar conhecimento físico para definir a relação entre os dados

Equação de Burgers

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

- Modelagem de gases compressíveis
- Ondas de choque
- Trânsito
- *Toy-problem* para Navier-Stokes

Equação de Burgers

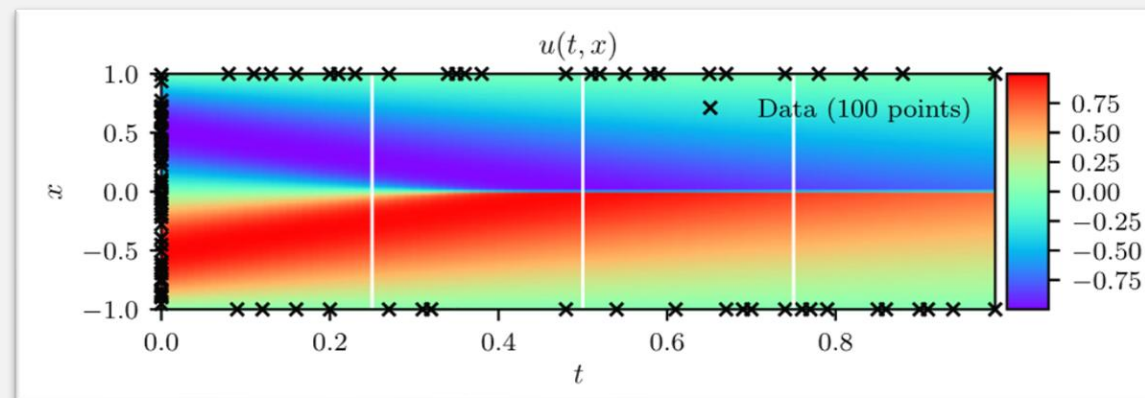
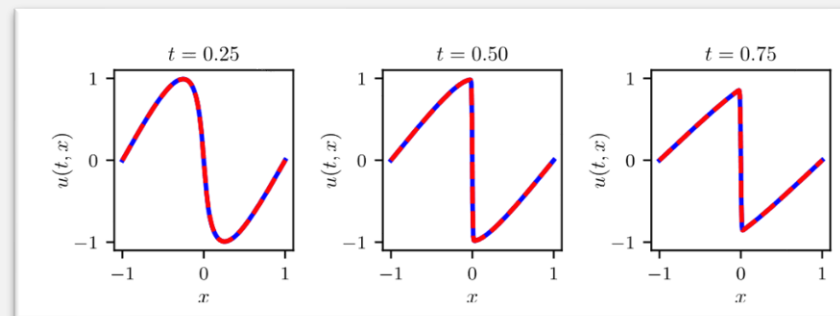


RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019.

Equação de Burgers

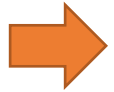
Como definir o problema?

- Equação governante
- Domínio
- Condição inicial
- Condições de contorno



Equação de Burgers

Como definir o problema?



Equação governante

- Domínio
- Condição inicial
- Condições de contorno

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

Equação diferencial parcial

$x, t \rightarrow$ Coordenadas

$u \rightarrow$ Função a ser resolvida ($u(t, x)$)

$\nu \rightarrow$ Coeficiente

Equação de Burgers

Como definir o problema?

- Equação governante
- ➔ Domínio
- Condição inicial
- Condições de contorno

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

Onde iremos resolver a equação

$x, t \rightarrow$ Coordenadas

$$0 \leq x \leq x_f = 1$$

$$0 \leq t \leq t_f = 0.5$$

Equação de Burgers

Como definir o problema?

- Equação governante
- Domínio
- ➔ Condição inicial
- Condições de contorno

Qual a situação no início da simulação?

$$u(t = 0, x) = f(x)$$

$$u(t = 0, x) = \sin(2\pi x)$$

Equação de Burgers

Como definir o problema?

- Equação governante
- Domínio
- Condição inicial



Condições de contorno

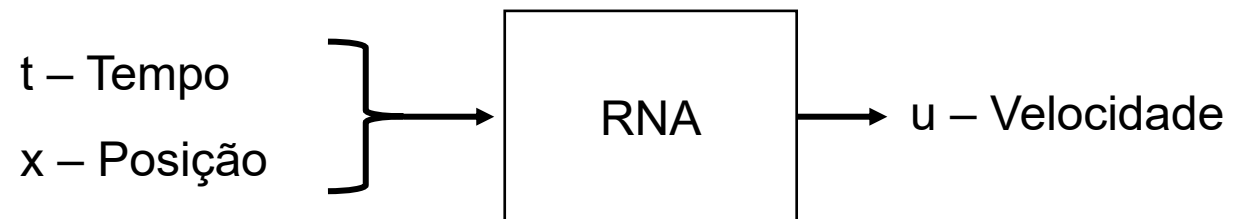
O que acontece nas beiradas do domínio?

$$u(t, x = 0) = g(t)$$

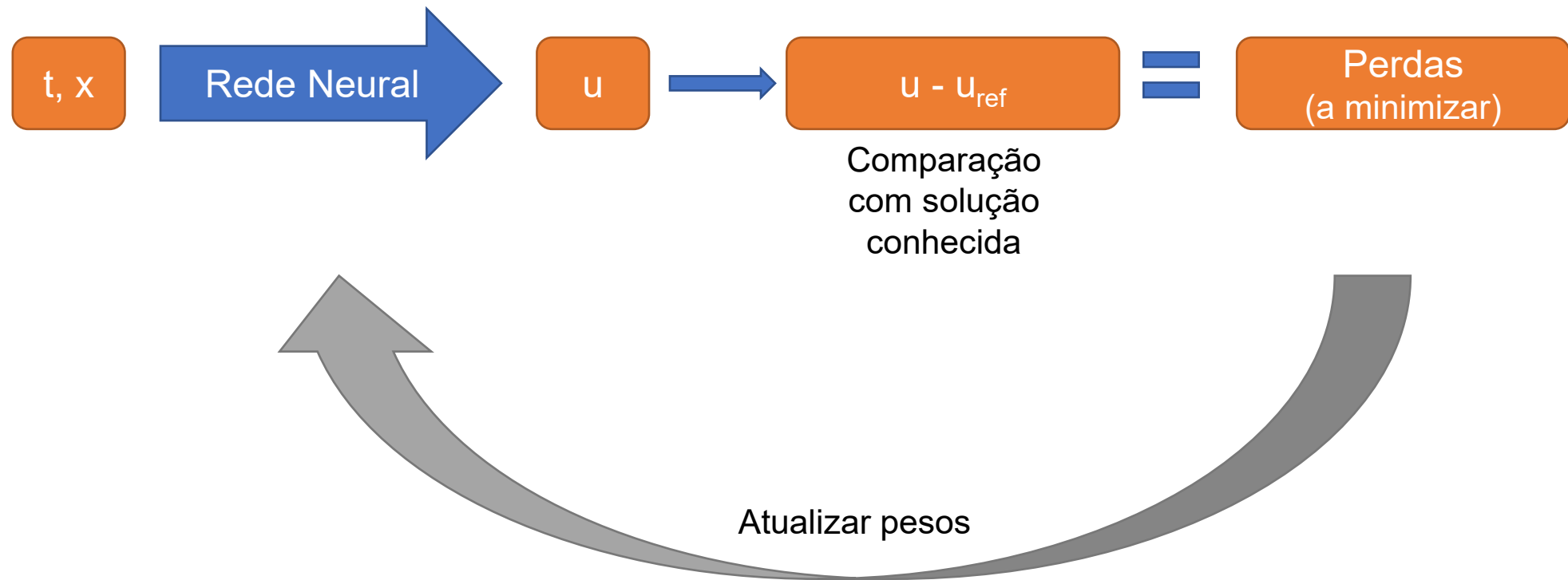
$$u(t, x = x_f) = h(t)$$

$$g(t) = h(t) = 0$$

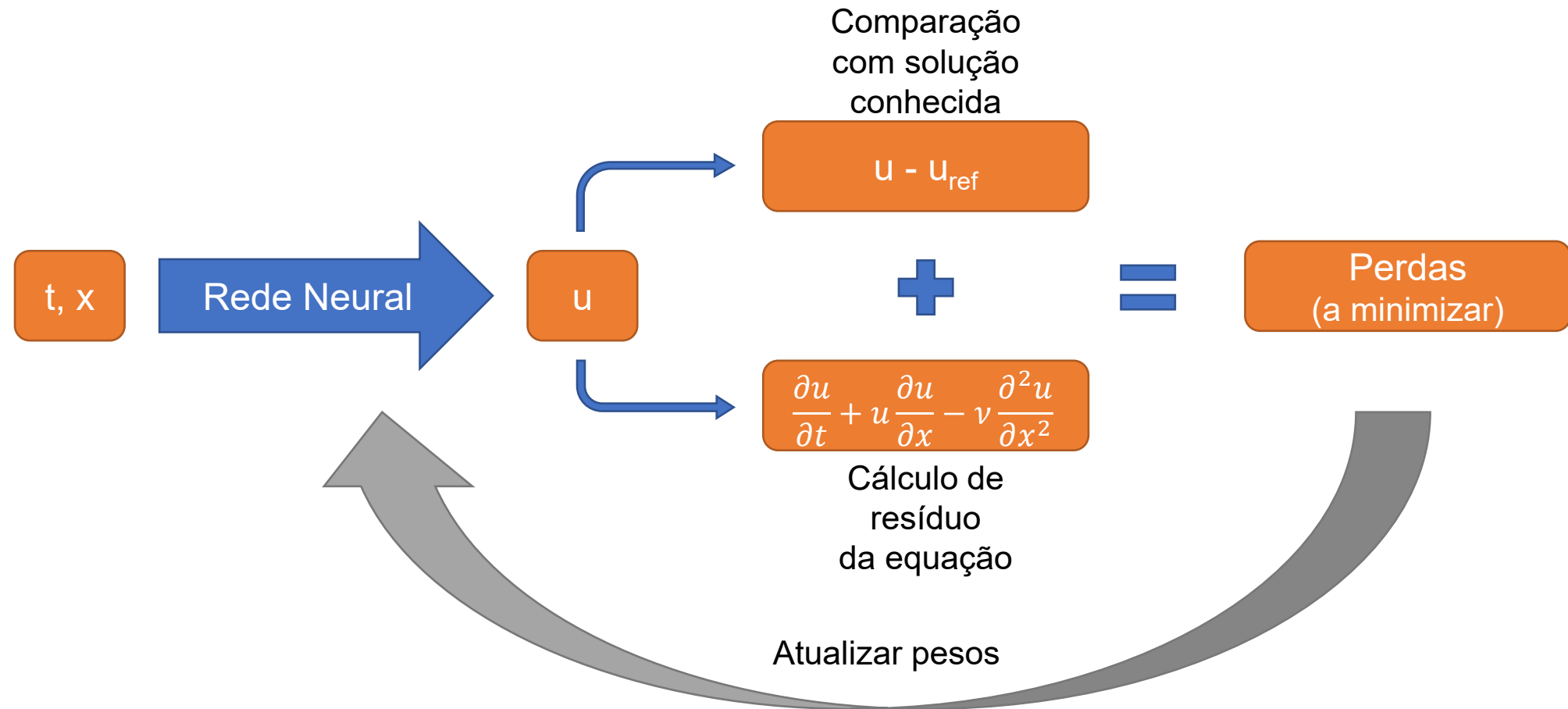
Onde queremos chegar?



Redes neurais treinadas com dados



Redes neurais treinadas com dados e física



Como calcular as derivadas parciais?

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = v \frac{\partial^2 u}{\partial x^2}$$

- Funcionalidade de derivadas automáticas
- PyTorch e TensorFlow

```
u_t = torch.autograd.grad(u, t, ...)
```

Outros pontos importantes

- Neste exemplo, não sabemos nada sobre a solução dentro do domínio
- Os pontos conhecidos ficam apenas nas condições iniciais e de contorno
- Também podemos incluir alguns pontos dentro do domínio

Outros pontos importantes

- Função de ativação

Podemos usar ReLU ou semelhantes?

Outros pontos importantes

- Equilíbrio entre as diferentes perdas

$$L = (1 - \alpha)L_{pontos} + \alpha L_{física}$$

Outros pontos importantes

Ressorteio dos pontos

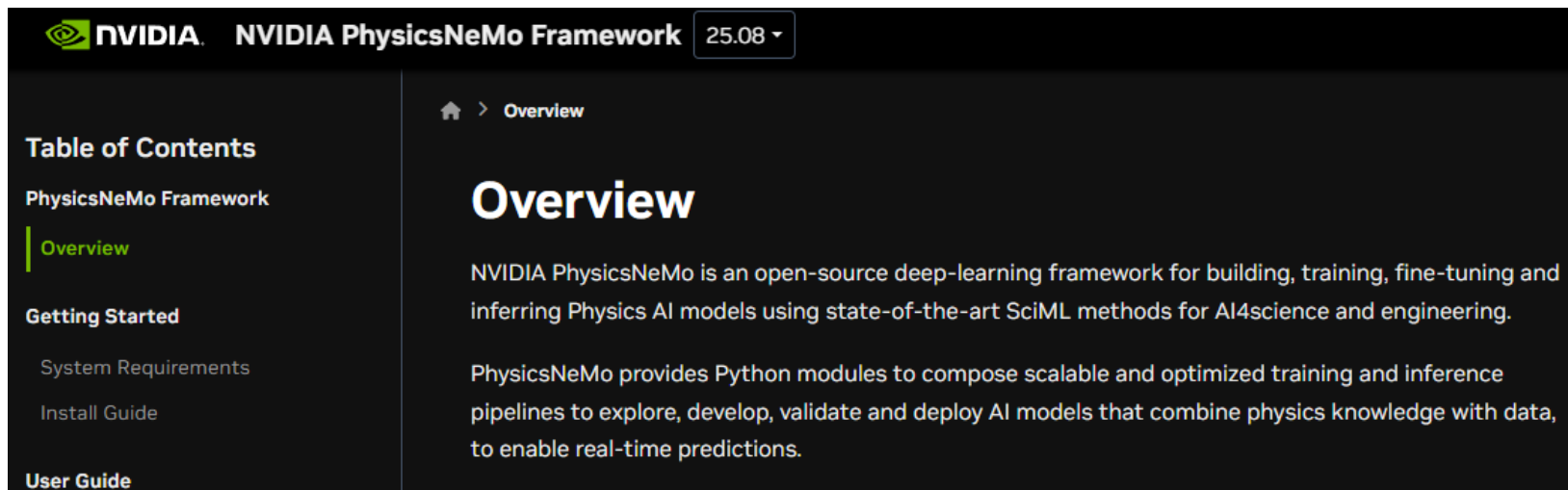
- Os pontos de avaliação da equação podem ser trocados infinitamente ao longo do treinamento
- Isso pode ser usado para reduzir o custo computacional total
- Também ajuda a evitar overfit

Implementação em Jupyter

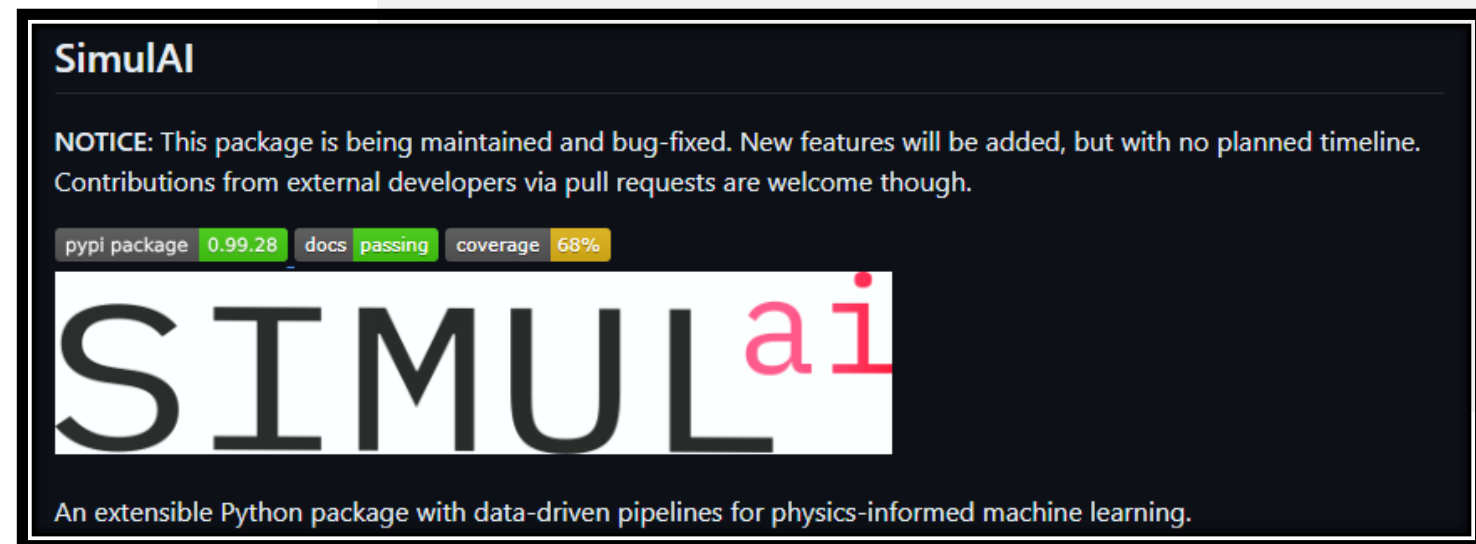
Ver arquivo no Moodle



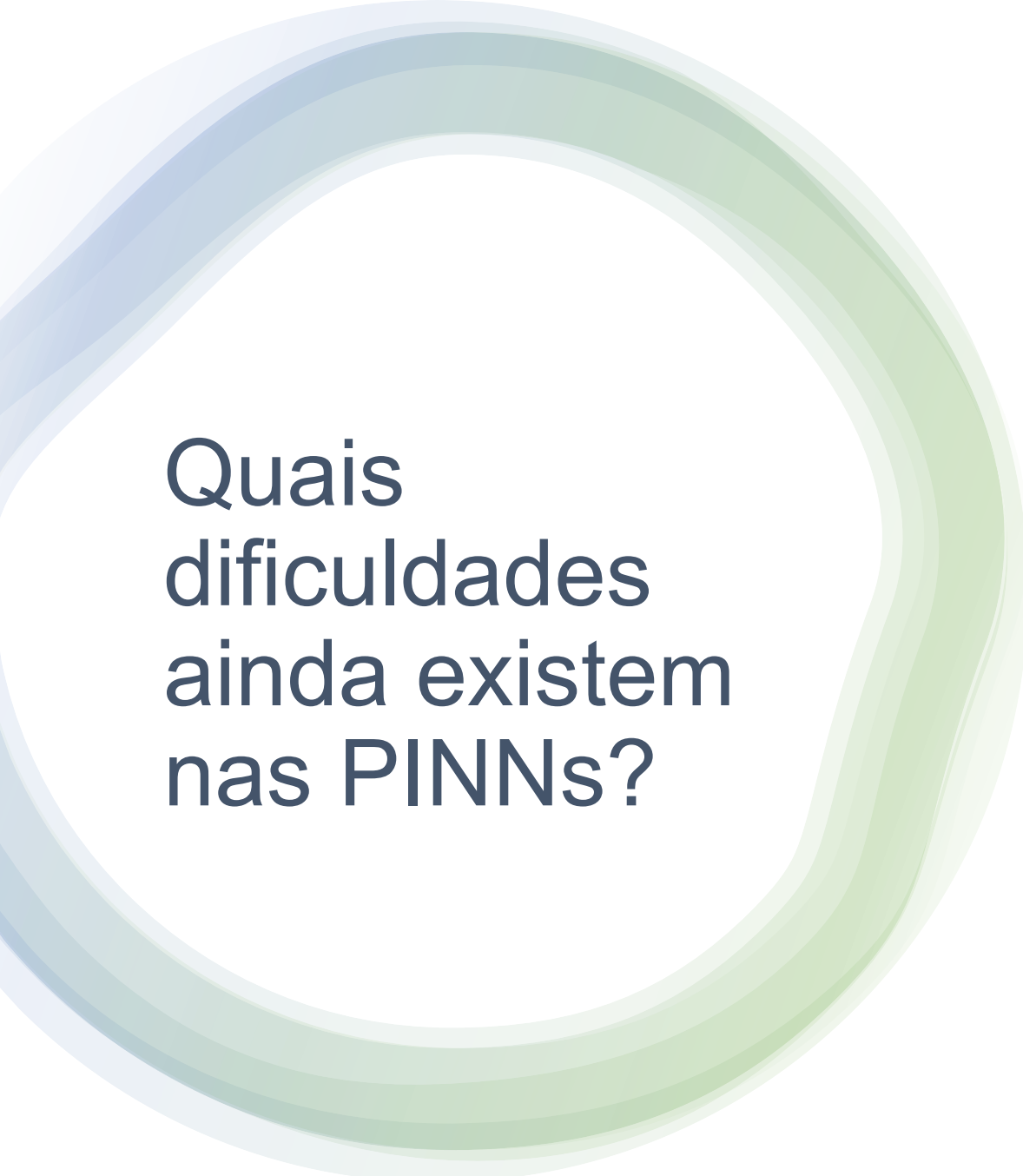
Bibliotecas



The screenshot shows the NVIDIA PhysicsNeMo Framework documentation page. The header includes the NVIDIA logo, the text "NVIDIA PhysicsNeMo Framework", and a version dropdown set to "25.08". The left sidebar contains a "Table of Contents" with links to "PhysicsNeMo Framework", "Overview" (highlighted), "Getting Started", "System Requirements", "Install Guide", and "User Guide". The main content area is titled "Overview" and describes the framework as an open-source deep-learning tool for building, training, fine-tuning, and inferring Physics AI models using state-of-the-art SciML methods. It also mentions that the framework provides Python modules for scalable and optimized training and inference pipelines.



The screenshot shows the SimulAI GitHub repository page. The title is "SimulAI". A notice states: "NOTICE: This package is being maintained and bug-fixed. New features will be added, but with no planned timeline. Contributions from external developers via pull requests are welcome though." Below the notice, there are badges for the package version (0.99.28), docs status (passing), and coverage (68%). The repository logo "SIMULai" is displayed, with "SIMUL" in black and "ai" in pink. At the bottom, a description reads: "An extensible Python package with data-driven pipelines for physics-informed machine learning."



Quais dificuldades ainda existem nas PINNs?

Soluções oscilatórias

Múltiplas escalas

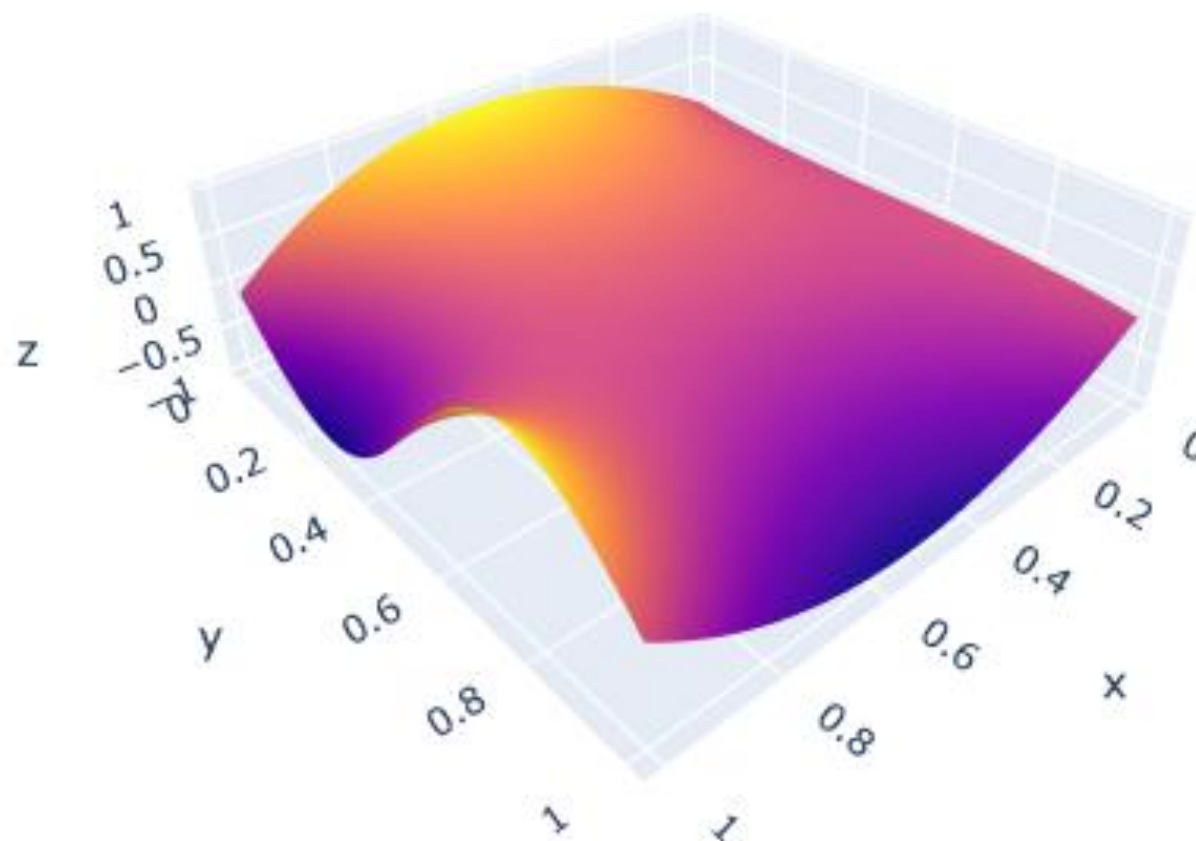
Caos

Mínimos locais

Equilíbrio entre equações de tipos diferentes

Troca de condições iniciais e de contorno

Troca de domínio



Trabalho 4

- Fazer a implementação de uma PINN

- Equação de Poisson 2D

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y)$$

- Domínio: $0 \leq x \leq 1, 0 \leq y \leq 1$

- Condições de contorno:
 $u(x, y = 0) = \sin(2\pi x)$

$$u(x, y = 1) = \sin(\pi x)$$

$$u(x = 0, y) = u(x = 1, y) = 0$$

- Diversos usos em vários campos, em geral se relaciona ao equilíbrio
- Superfície de uma bolha, tecido esticado, distribuição de pressão...